第4章 UNIX 文本编辑器

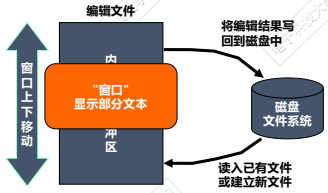
詹文翰

内容简介

- 一、文本编辑器的基础
- 二、nano 编辑器
- 三、vi 编辑器入门
- 四、vi 基本操作
- 五、vi 高级操作
- 六、vi 编辑器总结

一、文本编辑器的基础

- 文本编辑器 VS 字处理软件
  - 文本编辑器 -> 纯文本
  - 字处理软件 -> 排版和格式化
  - 常见文本编辑器：
    - 图形化：txt、subline
    - 命令行：nano、vi、emacs
  - 常见字处理软件：
    - word、pages
- 文本编辑器的基本工作原理



一、文本编辑器的基础

- UNIX中常见的文本编辑器：
  - ed：早期 UNIX 上的行编辑器。
  - ex：由 ed 扩展而来的 UNIX 经典行编辑器，vi 的前身。
    - 当进入 ex 的 visual 模式时，实质上就是进入了 vi 编辑器。
  - vi：由 ex 的 visual 模式发展而来的全屏编辑器，UNIX 上最常的用文本编辑器之一。
  - view：只读版本的 vi 编辑器。
  - vim：vi 的升级版， 提供了诸多方便的编辑功能和特性，如支持语法高亮、代码折叠、多级撤销等，还可以通过插件进行扩展。
  - emacs：和 vi 齐名的另一款超强文本编辑器，具有非常好的扩展性和定制性。
  - nano：一个轻量级文本编辑器，具有基本的编辑功能，且简单易用。

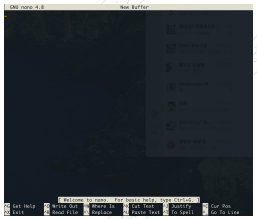
vi 系列编辑器

二、nano 编辑器

- nano 是 GNU 推出的一款“麻雀虽小、五脏俱全”的入门级文本编辑器，对初学者非常友好。
- nano 基本用法：
  - 启动软件：**nano [文件]**
  - 编辑文件：直接编辑即可（可使用方向键控制光标）
  - 读入文件：**<Ctrl-r>**
    - 将文件内容直接作为输入数据，输入到当前光标处。
  - 写出文件：**<Ctrl-o>**
    - 将编辑器中当前内容写入指定文件中。

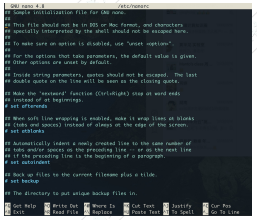
二、nano 编辑器

- nano 基本用法：
  - 剪切整行：**<Ctrl-k>**
    - 剪切光标所指的整行到剪切板。
  - 粘贴整行：**<Ctrl-u>**
    - 粘贴剪切板中的整行到光标处。
  - 查找内容：**<Ctrl-w>**
    - 查找指定内容，默认再次查找下一个。
  - 退出软件：**<Ctrl-x>**



二、nano 编辑器

- nano 配置文件：
  - 系统配置文件：/etc/nanorc
    - 系统配置文件兼作配置模版
  - 用户配置文件：~/.nanorc
- 常见配置项：
  - 自动缩进：set autoindent
  - 鼠标支持：set mouse
  - 取消帮助：set nohelp
  - 显示行号：set linenumbers
  - tab 大小：set tabsize 8



三、vi 编辑器入门

- 安装 vi
  - sudo apt install vim
- 启动 vi
  - 只读启动：
    - vi -R [文件]
    - view [文件]
  - 读写启动：
    - vi [文件]
  - 说明：
    - [文件] 可以是一个已存在文件（修改），也可以是一个不存在的文件（新建）。
    - 当不指明 [文件] 时，需要在 vi 运行时关联具体文件。

三、vi 编辑器入门

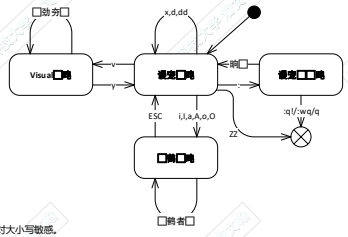
- vi 界面

三、vi 编辑器入门

- vi 工作模式
  - 命令模式：vi 的初始模式，主要包括移动光标、删除文本、粘贴文本和查找文本等。
    - 该模式下，所有键盘操作/操作组合均直接作用于文本，无需<回车>。
  - 输入模式：该模式下除 <ESC> 键以外的所有其它键都作为输入字符添加到被编辑的文本中。按下 <ESC> 键返回到命令模式。
  - 命令行模式：命令模式的子模式，在命令模式下点击“:”进入该模式，可与 vi 进行命令交互。按下 <ESC> 键返回到命令模式。
    - 该模式下，所有命令在输入完成之后，均以<回车>结束，指示 vi 开始执行命令。之后，返回到命令模式。
  - visual 模式：命令模式的子模式，在命令模式下点击“v”或者“V”进入该模式，可进行整块文本的选择。按下 <ESC> 键返回到命令模式。
    - 该模式下，可对选中的文本进行复制、删除等操作。之后，返回到命令模式。

三、vi 编辑器入门

- vi 工作模式
  - 模式转换状态机：


□动 作□
x,d,dd
Visual□□□
v
v
命令□□□
命令□□□□□□
:
命令□□□
ESC
i,I,a,A,o,O
:q!/:wq/q
ZZ
□□□□□□
□□□□□□

四、vi 基本操作

- 复制和粘贴（命令模式）
  - 在命令模式下，可以进行复制和粘贴操作。

命令	功能
yy	复制当前行到指定寄存器（即剪切板，默认为“号寄存器）
y+[Motion]	需要结合**[Motion]**一起使用，复制对应文本到指定寄存器（默认为“号寄存器）
P	将指定寄存器中的内容粘贴到当前光标之上（默认为“号寄存器）
p	将指定寄存器中的内容粘贴到当前光标之下（默认为“号寄存器）

  - 在命令行模式下，也可按行进行文本的复制，其格式如下：
    - :[Range]y <回车>
  - 实际上，前面的所有删除操作均可认为是剪切操作：
    - 删除操作所删除的内容均会被保存到指定寄存器中（默认为“号寄存器）。

四、vi 基本操作

- 撤销和重复（命令模式）
  - 在命令模式下，可以撤销和重复之前进行过的修改。

命令	功能
u	undo，撤销最近一次的修改，**本身不会被算作一次修改**
<Ctrl-r>	redo，撤销最近一次的撤销，**本身不会被算作一次修改**
.	重复最近一次的操作，**本身会被算作一次修改**
U	撤销所有最新的修改，**本身会被算作一次修改**，极少使用

  - 修改的定义：
    - 在 vi 中，命令模式下的**任一次**改变文本内容的**完整操作**被定义为一次修改。
    - 在 vi 中，在每次**进入输入模式**和**退出输入模式**之间的任何修改仅被定义为一次修改。
  - 撤销和重复的原理
    - vi 维护了一个修改队列，每次修改都会在队尾append一个元素。
    - u 和 <Ctrl-r> 在这个队列上回退或者前进
    - . 将队尾的元素再次append到队列

四、vi 基本操作

- 崩溃与恢复
  - 在使用 vi 进行文本编辑时，可能会遇到各类异常退出的情况（尤其是远程登录时）。好在，vi 编辑器提供了文件恢复功能：
    - 在编辑文件（如“filename”）时，vi 会在该文件所在目录下自动生成一个临时的隐藏文件“.filename.swp”，用于保存当前编辑文件的备份和编辑历史记录。
    - 当 vi 异常崩溃时，“.filename.swp”不会被正常删除，而是留在对应目录中。
    - 当 vi 再次编辑“filename”时，会发现“.filename.swp”的存在，从而确认之前在编辑该文件时发生过异常退出。此时，vi 便会询问用户如何进行处理。
    - 用户可以根据 vi 的提示选择恢复该文件，从而重新进入之前被中断的编辑状态；也可以选择不恢复该文件，从之前保存的原始版本开始重新编辑。
    - 用户恢复完文件之后，需要手动删除“.filename.swp”。

四、vi 基本操作

- 崩溃与恢复

```
ubuntu@i-0dff84f829ec31786:~/test$ ls -a
.  ..  .333.swp  333  initrd.img  kernel
ubuntu@i-0dff84f829ec31786:~/test$
```

```
E325: ATTENTION
Found a swap file by the name ".333.swp"
          owned by: ubuntu   dated: Fri Mar 24 09:55:56 2023
        file name: ~ubuntu/test/333
         modified: YES
         user name: ubuntu   host name: i-0dff84f829ec31786
        process ID: 85577
While opening file "333"
             dated: Fri Mar 24 10:07:26 2023

      NEWER than swap file!

(1) Another program may be editing the same file.  If this is the case,
    be careful not to end up with two different instances of the same
    file when making changes.  Quit, or continue with caution.
(2) An edit session for this file crashed.
    If this is the case, use "recover" or "vim -r 333"
    to recover the changes (see ":help recovery").
    If you did this already, delete the swap file ".333.swp"
    to avoid this message.

Swap file ".333.swp" already exists!
[O]pen Read-Only, (E)dit anyway, (R)ecover, (D)elete it, (Q)uit, (A)bort:
```


小练习：

- 尽量在命令模式完成以下操作：
  - 在HOME目录下编辑 UNIX.txt 文件；
  - 输入文本：“This is **the** UNIX vi practice file.”
  - 在这行之前新起一行，输入文本：“Hello, everyone!”
  - 将前面的“the”替换为“an”；
  - 复制“Hello, everyone!”并粘贴到文件末尾；
  - 删除“Hello”，将其改为“goodbye”；
  - 将“goodbye”改为“Goodbye”；
  - 删除“Goodbye”所在整行；
  - 删除所有文本；
  - 输入“No content!；
  - 保存退出。

五、vi 高级操作

- 大范围移动光标
- 组合命令
- 寄存器使用
- 搜索和替换
- 定制 vi 编辑器
- 宏录制和使用
- 缓冲区、窗口和标签
- 其它常用命令

五、vi 高级操作

- 大范围移动光标
  - 除前面介绍的移动光标操作之外，对于大型文件，还可使用以下大范围移动光标操作。
  - 命令模式下：

按键	功能
<Ctrl-d>	向前翻半页，行数可调（scroll参数）
<Ctrl-u>	向后翻半页，行数可调（scroll参数）
<Ctrl-f>	向前翻页
<Ctrl-b>	向后翻页
gg	光标移动到文件的第一行
G	光标移动到文件的最后一行
数字+G	光标移动到 数字 所对应的行号
%	光标在括号、大括号和方括号之间跳转
<Ctrl-g>	在状态栏显示当前光标的位置

  - 命令行模式下：
    - :数字　　定位到第 数字 行
    - :0 或 :1　　定位到第1行的行首
    - :$　　定位到最后一行的行首

五、vi 高级操作

- 组合命令
  - vi 有自己的一套命令构造语法，用于生成更为复杂的组合命令，具体构造方法如下：
    - 组合命令 = 操作符 + [Motion]
    - 组合命令 = [数字] + 操作符
    - 组合命令 = [数字] + [Motion]
    - 组合命令 = [数字] + 操作符 + [Motion]
  - 其中：
    - 操作符指前面学到的一些针对文本的动词，常见的有：d、c、y等。
    - [Motion] 指的是移动光标的命令，常见的有：0、$、w、e、b等。
    - [数字] 指需要重复其后面跟着的操作的次数。
  - 举例：
    - dw、d$、4yy、2dw

五、vi 高级操作

- 寄存器使用
  - vi 设计了特别的内容暂存机制——寄存器——用于保存拷贝和删除的文本
  - 在命令模式下以“双引号开头可以引用这些寄存器
  - 在命令行模式下输入 :reg [寄存器号] <回车> 可以显示指定寄存器中的内容
    - 若不指定 [寄存器号]，则显示所有寄存器中的内容
  - 寄存器介绍：
    - 有名寄存器a-z，一共26个，供用户自行使用
    - 编号寄存器0-9，一共10个，vi 自己使用，存放复制和删除的文本内容
      - 0号寄存器保存的是最近拷贝的文本内容
      - 1-9号寄存器保存的是最近用删除的文本内容

五、vi 高级操作

- 寄存器使用
  - 缺省寄存器
    - ”　　最近的复制和删除的文本内容
    - -　　最近进行的小修改
    - .　　最近插入的文本
    - :　　命令行模式最近输入的命令
    - %　　当前窗口对应的文件名
    - /　　最近查找的文本
  - 寄存器默认行为
    - dd命令将删除行拷贝到 “” 和 “1寄存器，并且依次下压2-9号寄存器
    - yy命令将复制行拷贝到 “” 和 “0寄存器
    - p或P命令将缺省寄存器 “” 的内容拷贝到光标位置

```
:registers
Type Name Content
  l  ""   f3^J
  l  "0   7^J
  l  "1   f3^J
  l  "2   ^J
  l  "3   ^J
  l  "4   ^J
  l  "5   croll^J
  l  "6   visualbell^J
  l  "7   showmode^J
  l  "8   ignorecase^J
  l  "9   expandtab^J
  c  "a   ^[^[^[^[^[^[^[^[^[^[^[^[^[<80><fd>aa^[^[^[^[^[^[^[^[a^[^[^[
  c  "p   <80><ku><80><kd><80><kr><80><kr>asdsfdff^[<80><fd>a
  c  "u   uu<80><ku><80><ku><80><ku><80><ku><80><ku><80><ku><80><ku><80><kd-
  c  "w   xx
  c  "-
  c  "%   f1
  c  "/   \<cat\>
Press ENTER or type command to continue_
```


五、vi 高级操作

- 寄存器使用
  - visual 模式
    - 在命令模式下按 v 进入 visual 模式
    - 在 visual 模式下，使用khjl或方向键移动光标，选中内容
    - y命令将选择内容拷贝到 “” 和 “0寄存器
    - d或x命令将选择内容**剪切**到 “” 和 “1寄存器，并且依次下压2-9号寄存器
  - 有名寄存器供用户自行使用，如：
    - “wdd　　剪切当前行到 “w寄存器
    - “z7yy　　拷贝从当前光标开始算，一共7行，到” z寄存器
  - 使用 “寄存器+p 或者 “寄存器+P 将寄存器内容粘贴到光标位置，如：
    - “zp　　粘贴 “z寄存器中的内容到光标处
    - “5P　　粘贴 “5号寄存器中的内容到光标处

五、vi 高级操作

- 搜索和替换——搜索
  - 在命令模式下，可以通过/和? 分别实现向后和向前搜索：

命令	功能
/	向后搜索
?	向前搜索
n	按照/或?的方向搜索下一个匹配项
N	按照/或?的反方向搜索下一个匹配项

- Tips：
  - 现代 vi 编辑器的搜索支持正则表达式。

五、vi 高级操作

- 搜索和替换——替换
  - 自动替换
    - 可以在命令行模式下，通过输入替换命令（s）完成文本的自动替换，基本格式为：
      - :[Range]s/old/new[/g]<回车>
    - 其中:
      - [Range] 为可选，表示行范围（前面已介绍），没有该字段表示光标所在行。
      - [/g] 为可选，代表是否在 [Range] 范围内进行全部替换。
    - 举例:
      - :s/old/new　　搜索当前行，将匹配的第一个old字符串替换为new字符串
      - :s/old/new/g　　搜索当前行，将匹配的所有old字符串替换为new字符串
      - :3,9s/old/new　　搜索3-9行的内容，将匹配的第一个old字符串替换为new字符串
      - :%s/old/new　　搜索所有行的内容，将匹配的第一个old字符串替换为new字符串
      - :%s/old/new/g　　搜索所有行的内容，将匹配的所有old字符串替换为new字符串

五、vi 高级操作

- 搜索和替换——替换
  - 逐个替换
    - 在某些情况下，用户可能需要逐个确认要替换的文本内容，并在确定需要时再进行替换。类似于word中的 “查找下一个/替换” 操作模式。
    - 可以在命令模式下，通过一系列命令实现文本的逐个替换（将old替换为new）：
      - **/old<回车>**　　搜索到第一个old
      - **cw**new**<ESC>**　　替换单词old为new，并退回命令模式
      - **n**　　搜索到下一个old
      - .　　重复替换操作
      - ……　　就此往复，按需替换

五、vi 高级操作

- 定制 vi 编辑器——环境选项
  - vi 编辑器包含很多环境设置选项，通过设置或取消某些选项，用户可以对编辑环境进行个性化定制。
  - 可以通过命令行模式下的 set 命令进行这些选项的设置，用法如下：
    - :set all　　显示所有选项
    - :set　　显示被修改过（非默认值）的选项
    - :set X?　　显示选项X的值
  - vi 编辑器的环境选项分为以下三个类型：
    - 布尔型：X、noX
    - 数值型：X=数字
    - 字符串：X=字符串

```
:set nu              //设置编辑时显示行号
:set nonu            //表示编辑时不显示行号
:set shiftwidth?     //查看当前的缩进字符数
shiftwidth=8
:set helplang?       //查看帮助语言
helplang=en
```


五、vi 高级操作

- 定制 vi 编辑器——环境选项
  - 常用选项：

选项	缩写	类型	功能
number	nu	bool	显示行号
autoindent	ai	bool	开启缩进，可用<Ctrl-d>减少一级缩进
shiftwidth	sw	num	执行缩进或反缩进操作时移动的空格数量
tabstop	ts	num	制表符显示在屏幕上的宽度
softtabstop	sts	num	制表符在输入时实际得到的宽度，通常设置为-1，表示和shiftwidth一致
expandtab	et	bool	输入模式下，将制表符替换为对应数目的空格
ignorecase	ic	bool	搜索的时候忽略大小写
showmode	sm	bool	在屏幕显示编辑器的当前模式
visualbell	vb	bool	在屏幕上闪烁一下以代替出错声音提示
scroll		num	设置<Ctrl-d>、<Ctrl-u>翻页的行数
magic		bool	在搜索中启用正则表达式符号
foldenable		bool	允许对编辑文本进行代码折叠，设置后，可使用zo/zc命令打开/关闭折叠
foldmethod		string	对文本进行折叠的依据，通常设置为 “indent”

五、vi 高级操作

- 定制 vi 编辑器——缩写
  - 通过**命令行模式**下的 ab 命令，可以创建出**输入模式**下的缩写 “快捷键” 。让用户只需使用缩写便可以快速输入完整文本。
  - ab 命令使用方法：
    - :ab uno UNIX OS<回车>　　设置 uno 为字符串 UNIX OS 的缩写
    - :ab<回车>　　显示所有生效的缩写
    - :unab uno <回车>　　取消之前设置的 uno 的缩写
  - 在**输入模式**下，输入 uno<空格> 即可自动扩展为 UNIX OS。
    - 若在uno后紧跟的不是空格，而是其它字符，则不会自动扩展。

五、vi 高级操作

- 定制 vi 编辑器——映射
  - 映射可用于将**按键序列**映射为**单个按键**。
  - 通过**命令行模式**下的 map 或者 noremap 系列命令，可以创建映射。
  - map 命令使用方法
    - :map <任意键> 任意键序列<回车>　　设置映射
    - :map<回车>　　显示所有生效的映射
    - :unmap <映射键>　　取消之前指定的映射（包括所有 map 和 noremap 系列命令）
  - 举例：
    - :map <F1> gg　　将<F1>键映射为gg
    - :map <C-v> <F1>　　将<Ctrl-v>键映射为<F1>键，支持递归映射
    - :map <F2> :set nu<cr>　　将<F2>键映射为命令行命令

五、vi 高级操作

- 定制 vi 编辑器——映射
  - map 和 noremap 的区别：
    - map 会进行递归映射，noremap 不会进行递归映射。因从，noremap 通常比 map 更安全。
  - map 系列：
    - nmap：仅针对命令模式的 map
    - imap：仅针对输入模式的 map
    - cmap：仅针对命令行模式的 map
    - vmap：仅针对 visual 模式的 map
  - noremap 系列同理：
    - **nnoremap**、inoremap、cnoremap、vnoremap
  - Tips：
    - 在 vi 中，键盘上预留的没有任何功能的按键很少。因此，建议使用功能键（<F1>-<F12>）进行映射。

五、vi 高级操作

- 定制 vi 编辑器——配置文件
  - 通过命令行模式下进行的 vi 环境设置并不能永久生效，vi 退出即失效。可通过.exrc或.vimrc文件让用户设置永久生效。
  - 当用户启动 vi 时，vi 会自动搜索用户的主目录下是否存在.exrc文件（vim还另外再支持.vimrc文件）。vi 会根据该文件内容设置编辑环境。
  - 通常，.vimrc 文件的优先级较 .exrc 文件更高。

```
syntax on

set number
set autoindent
set shiftwidth=4
set tabstop=4
set softtabstop=-1
set expandtab
set novisualbell
set scroll=10

nnoremap <F1> gg

~
~
~
".exrc" 13L, 151C
```


五、vi 高级操作

- 宏录制和使用
  - vim 提供了宏让用户能够录制一些较为复杂的操作序列（存放到指定寄存器中），之后便可以按需执行这些操作序列。
  - 宏录制（命令模式）
    - q寄存器号　　开始宏录制，状态栏会显示 recording @寄存器号，建议用有名寄存器
    - 自由操作　　vi 会进行录制
    - q　　在命令模式下再次按下 q 以结束录制
  - 查看录制的宏（命令行模式）
    - :reg 寄存器号　　会显示出记录的命令序列，部分命令存在特殊显示
  - 使用录制的宏（命令模式）
    - [计数]@寄存器号　　将 寄存器号 中保存的操作序列重复执行 [计数] 次

五、vi 高级操作

- 宏录制和使用
  - 举例：
    - 命令模式下
      - qw　　使用“w寄存器开始宏录制，状态栏会显示 recording @w
      - xx　　删除两个字符
      - q　　结束录制
    - 命令行模式下
      - :reg w　　查看“w寄存器中的宏
    - 命令模式下
      - 10@w　　重复执行“w寄存器中的宏10次，即执行20次 x 操作

五、vi 高级操作

- 缓冲区、窗口和标签——缓冲区
  - 缓冲区（buffer）：保存所编辑文件数据的内存空间。当用户在 vi 中打开一个文件时，文件的数据就与一个缓冲区绑定。
    - 若同时打开三个文件，就会有三个缓冲区分别与这三个文件对应。
  - 示例：
    - 同时打开多个文件，产生多个缓冲区
      - 在 shell 界面下直接执行 vi f1 f2 f3
      - 在命令行模式下，通过 :e 命令打开多个文件 :e 文件名<回车>
    - 查看所有缓冲区 :ls
    - 在缓冲区之间切换
      - :bnext 或者 :bprevious
    - 退出所有缓冲区 :qa 或者 :qa!

五、vi 高级操作

- 缓冲区、窗口和标签——窗口
  - 窗口（window）：在缓冲区上的一个视口。vi 具备多窗口显示能力。
    - 一个缓冲区可对应多个窗口；一个窗口需要绑定某个缓冲区（可以切换）。
  - 示例：
    - 在 vi 中打开多个窗口（:split/:vsplit 命令）
      - :split<回车>　　在当前缓冲区上打开一个新的横向窗口
      - :vsplit [文件名]<回车>　　在新的对应[文件名]的缓冲区上打开一个新的纵向窗口
    - 在窗口之间跳转
      - <Ctrl-w>h　或者　<Ctrl-w><方向左键>
      - <Ctrl-w>j　或者　<Ctrl-w><方向下键>
      - <Ctrl-w>k　或者　<Ctrl-w><方向上键>
      - <Ctrl-w>l　或者　<Ctrl-w><方向右键>
    - 改变当前窗口大小
      - <Ctrl-w>[计数]+　增加窗口 [计数] 行数
      - <Ctrl-w>[计数]-　减少窗口 [计数] 行数
      - <Ctrl-w>[计数]>　增加窗口 [计数] 列数
      - <Ctrl-w>[计数]<　减少窗口 [计数] 列数
    - 退出窗口 :q :q! :qa :qa!

五、vi 高级操作

- 缓冲区、窗口和标签——标签
  - 标签（tab）：是一个多窗口的集合。可将多个窗口归属于一个标签。
  - 示例：
    - 在 vi 中打开一个新的标签
      - :tabnew [文件名]<回车>　　打开一个新标签，标签内包含一个新窗口，窗口对应[文件名]的缓冲区
    - 在标签之间跳转
      - 命令行模式
        - :tabnext 或者 :tabprevious
        - :tablast 或者 :tabfirst
      - 命令模式
        - gt　　切换到下一个标签
        - gT　　切换到上一个标签
    - 退出标签 :tabclose :q :q! :qa :qa!

五、vi 高级操作

- 缓冲区、窗口和标签——三者关系
  - 窗口和标签的关系类似于三维坐标：
    - 窗口对应X、Y轴：可在XY平面任意layout。
    - 标签对应Z轴：可以对应一系列XY平面上的layout。
  - 每打开一个文件对应一个缓冲区，可以有多个窗口和一个缓冲区关联。

五、vi 高级操作

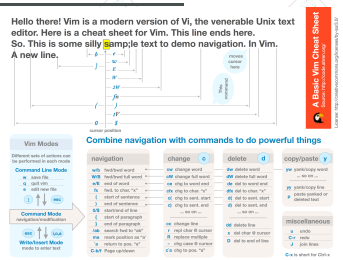
- 其它常用命令——命令行模式

命令	功能
:[Range]w [文件名]<回车>	将[Range]所指的内容存储到[文件名]中
:[Range]w! [文件名]<回车>	将[Range]所指的内容强制存储到[文件名]中
:r 文件名<回车>	读入 文件名 中的内容到光标位置
:shell<回车>	切换到shell，退出shell后回到 vi
:!shell命令<回车>	在 vi 中执行 shell命令，结果显示在状态栏中
:r!shell命令<回车>	执行 shell命令，并将命令输出粘贴到光标位置

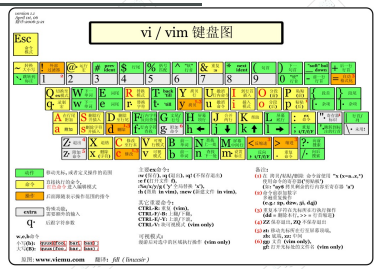
- 其它常用命令——命令模式

命令	功能
J	把下一行连接到当前行的行尾
>>	缩进
<<	反缩进

六、vi 编辑器总结



六、vi 编辑器总结



第四章小结

- 文本编辑器的基础
- nano 编辑器
- vi 编辑器入门
- vi 基本操作
- vi 高级操作
- vi 编辑器总结